

Indian Institute of Technology Dharwad

CS214: Artificial Intelligence Laboratory

Lab 11: Autoregression

Problem Statement

You are given the dataset `AAPL_historical_data.csv`, which contains historical stock market data of apple recorded on a daily basis. The dataset spans from 12-12-1980 to 13-12-2024 and is indexed by date and includes the following attributes: `Date`, `Open`, `High`, `Low`, `Close`.

The objective of this task is to build an autoregression (AR) model and a neural network model for univariate time series analysis. The models capture the temporal relationships in stock prices. The goal is to model and predict the stock's closing price using its past values, where the date is represented on the x-axis and the closing price on the y-axis.

1. Autocorrelation line plot with lagged values

- (a) Create a line plot with the x-axis as the index of the date and the y-axis as the closing price of the stock, as shown in Figure 1. Observe the overall trends, major peaks, and significant drops in the stock price over time.
- (b) Generate another time sequence with one-day lag to the given time sequence. Find the Pearson correlation (autocorrelation) coefficient between the generated one-day lag time sequence and the given time sequence.
- (c) Generate a scatter plot between the given time sequence and the one-day lagged generated sequence in 1.(b). What do you infer regarding correlation? Does it match with the computed correlation coefficient in 1.(b)?
- (d) Plot a correlogram or Auto Correlation Function using the Python inbuilt function `plot_acf` up to 600 lag values. Observe the trend in the line plot with an increase in lagged values.

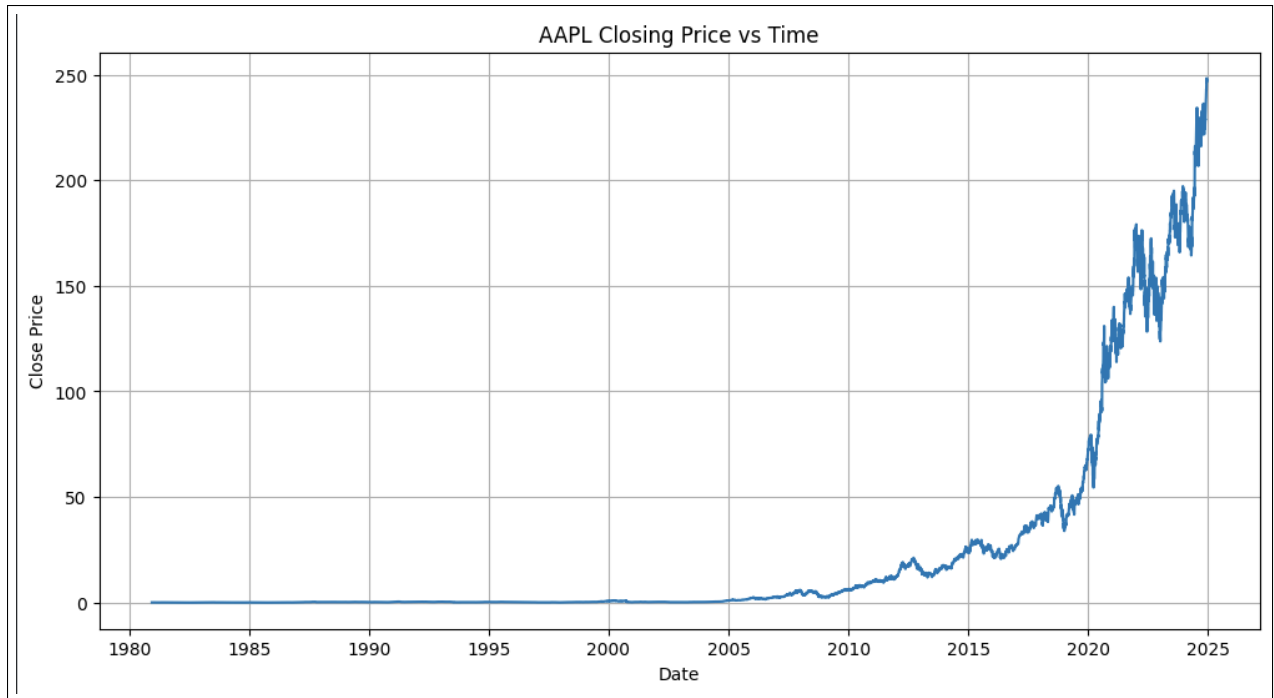


Figure 1: Line plot of closing stock price vs time

2. Auto Regression Model

- (a) Split the data into two parts. Use the first 10,000 observations of the sequence as the training dataset and the remaining observations as the test dataset. (You may use slicing operation for the same to maintain the order of the sequence. Note that, you should not shuffle randomly.) Plot the train and test sets.

Generate an autoregression (AR) model using the `AutoReg()` function from the `statsmodels` library. This function generates an AR model with the specified training data and lagged values (given as its input). Use 5 lagged values as its input ($p = 5$). Train/Fit the model onto the training dataset. Obtain the coefficients ($w_0, w_1, \dots, w_p$) from the trained AR model.

- (b) Using these coefficients, predict the closing price values (using the relation given above) for the test dataset. Note that, you have to make a 1-step ahead prediction at each time step.
 - i. Give a scatter plot between actual and predicted closing price values.
 - ii. Give a line plot showing actual and predicted closing price values for the test dataset over time.
 - iii. Compute Root Mean Square Error (RMSE) and Mean Absolute Percentage Error (MAPE) between actual and predicted closing price values.

3. AR Models with Different Lags

Generate AR models using the `AutoReg()` function with lag values of 1, 5, 10, 50, 100, 150, 200, 250, 300, 350, 400, 450, 500, 550 and 600 days.

- (a) Give a line plot showing actual and predicted closing price values for the test dataset over time.
- (b) Compute the RMSE and MAPE between predicted and actual closing price values in each case. Give a bar chart showing RMSE on the y-axis and lagged values on the x-axis. Also, give a bar chart showing MAPE on the y-axis and lagged values on the x-axis. Infer the changes in RMSE and MAPE with changes in lagged values.

4. Optimal Lag Selection

Compute the heuristic value for the optimal number of lags up to the condition on autocorrelation such that $|\text{AutoCorrelation}| > \frac{2}{\sqrt{T}}$, where T is the number of observations in the training dataset. Use it as input in the `AutoReg()` function to predict the stock's closing price on a daily basis and compute the RMSE and MAPE values. Compare this result with that of Question 3.

5. Neural Network Model

Part A: Single Hidden Layer Neural Network Architectures

Perform neural network-based regression and performance evaluation as listed below.

1. Load the dataset `AAPL_historical_data.csv` and construct lagged input representations using lag values of 5 and 600, where the previous p closing prices are used to predict the next closing price.
2. Split the dataset into training and test sets using the same split as defined in Question 2.
3. Implement fully connected neural network models using PyTorch with the following specifications:
 - Input layer: p neurons (corresponding to lagged input features, where $p \in \{5, 600\}$).
 - Activation function: `sigmoid`
 - Output layer: 1 neuron (predicted closing price).
4. Design and experiment with the following architectures for each lag value:
 - (a) **Single hidden layer architectures for lag = 5:**
 - Architecture 1: Input layer $\rightarrow$ hidden layer (64 neurons) $\rightarrow$ output layer

- Architecture 2: Input layer $\rightarrow$ hidden layer (128 neurons) $\rightarrow$ output layer
 - Architecture 3: Input layer $\rightarrow$ hidden layer (256 neurons) $\rightarrow$ output layer
- (b) **Single hidden layer architectures for lag = 600:**
- Architecture 1: Input layer $\rightarrow$ hidden layer (512 neurons) $\rightarrow$ output layer
 - Architecture 2: Input layer $\rightarrow$ hidden layer (1024 neurons) $\rightarrow$ output layer
 - Architecture 3: Input layer $\rightarrow$ hidden layer (2048 neurons) $\rightarrow$ output layer
5. Train the neural network models using:
 - Optimizer: Stochastic Gradient Descent (SGD)
 - Loss function: Mean Squared Error (MSE)
 6. Plot the training loss versus epochs for all architectures and for both lag values.
 7. For each lag value, evaluate all models using the test dataset and compute the following metrics:
 - Root Mean Square Error (RMSE)
 - Mean Absolute Percentage Error (MAPE)
 8. Plot the actual vs predicted closing price values for all neural network architectures for both lag values (lag = 5 and lag = 600).
 9. Compare the performance across different architectures and between the two lag values using RMSE and MAPE. Based on the results, identify the best-performing neural network model.
 10. Finally, compare the performance of the best neural network model with the AR model (lag = 5 and lag=600) from Question 3 based on RMSE and MAPE. Comment on the differences in performance.

Functions / Code Snippets

```
import pandas as pd
from statsmodels.tsa.ar_model import AutoReg as AR
from statsmodels.graphics.tsaplots import plot_acf

# Load dataset
df = pd.read_csv("apple_stock_data.csv", parse_dates=["Date"])
df.set_index("Date", inplace=True)

# Example: Lag-1 autocorrelation
df["lag1"] = df["Close"].shift(1)
corr = df[["Close", "lag1"]].corr().iloc[0,1]

# Example: ACF plot
plot_acf(df["Close"].dropna(), lags=60)

# Train-test split
train = df.iloc[:10000]
test = df.iloc[10000:]

# AR model (lag = 5)
model = AR(train["Close"], lags=5).fit()
coef = model.params

# Hint: Use coefficients to compute predictions
# yhat = w0 + w1*x(t) + ... + wp*x(t-p+1)
```

Neural Network Implementation for Time Series Prediction

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

# -----

# Load dataset

# -----

df = pd.read_csv("daily_covid_cases.csv", parse_dates=["Date"])
```

```

df = df.sort_values("Date")

series = df["Close"].values

# -----

# Create lagged dataset (lag = 10)

# -----

def create_lagged_data(series, lag=5):
X, y = [], []
for i in range(lag, len(series)):
X.append(series[i-lag:i])
y.append(series[i])
return np.array(X), np.array(y)

lag = 10
X, y = create_lagged_data(series, lag)

# -----

# Train-test split

# -----

X_train, X_test = X[:-5000], X[-5000:]
y_train, y_test = y[:-5000], y[-5000:]

# -----

# Normalization

# -----

scaler_X = StandardScaler()
scaler_y = StandardScaler()

X_train = scaler_X.fit_transform(X_train)
X_test = scaler_X.transform(X_test)

y_train = scaler_y.fit_transform(y_train.reshape(-1,1)).ravel()
y_test_scaled = scaler_y.transform(y_test.reshape(-1,1)).ravel()

# Convert to tensors

X_train_t = torch.tensor(X_train, dtype=torch.float32)

```

```

y_train_t = torch.tensor(y_train, dtype=torch.float32).view(-1,1)
X_test_t = torch.tensor(X_test, dtype=torch.float32)

# -----

# Neural Network Model

# -----

class NNModel(nn.Module):
    def __init__(self, input_size, hidden_layers):
        super(NNModel, self).__init__()

        '''
            layers = []
            prev_size = input_size

            for h in hidden_layers:
                layers.append(nn.Linear(prev_size, h))
                layers.append(nn.Tanh())
                prev_size = h

            layers.append(nn.Linear(prev_size, 1))
            self.net = nn.Sequential(*layers)

        def forward(self, x):
            return self.net(x)
        '''

# -----

# Training function

# -----

def train_model(model, X, y, epochs=200, lr=0.01):
    criterion = nn.MSELoss()
    optimizer = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9)

    '''
    losses = []

    for epoch in range(epochs):
        model.train()

        optimizer.zero_grad()
        outputs = model(X)

```

```

        loss = criterion(outputs, y)
        loss.backward()
        optimizer.step()

        losses.append(loss.item())

return model, losses
'''

# -----

# Evaluation function

# -----

def evaluate_model(model, X, y_true):
    model.eval()

    '''
    with torch.no_grad():
        preds = model(X).numpy()

    preds = scaler_y.inverse_transform(preds).ravel()

    rmse = np.sqrt(np.mean((y_true - preds)**2))
    mape = np.mean(np.abs((y_true - preds) / y_true)) * 100

    return preds, rmse, mape
    '''

# -----

# Train model (example: single layer)

# -----

model = NNModel(input_size=5, hidden_layers=[64])
model, losses = train_model(model, X_train_t, y_train_t)

# -----

# Predictions

# -----

preds, rmse, mape = evaluate_model(model, X_test_t, y_test)

```

```
print("RMSE:", rmse)
print("MAPE:", mape)

# -----

# Plot: Actual vs Predicted

# -----

plt.figure(figsize=(10,5))
plt.plot(y_test, label="Actual")
plt.plot(preds, label="Predicted")
plt.xlabel("Time Index")
plt.ylabel("Close Price")
plt.title("Actual vs Predicted (Neural Network)")
plt.legend()
plt.show()
```

Note

Please upload the completed Jupyter Notebook. Copy all Jupyter Notebook and data files in a folder and rename the folder as your rollnumber_Lab11, then create the zip file with the name your rollnumber_Lab11.zip. Finally, upload the zip file on the Moodle for evaluation.